APPENDIX 1

Prompts and Model Settings

LLM-assisted systematic review screening pipeline

Item	Implementation captured in this appendix
General article screening	SYSTEM_PROMPT plus one formatted metadata block per article
Title-only screening	Same prompt; abstract field masked as <Missing>
Title-abstract screening	Same prompt; title, authors, year, and abstract supplied
Full-text PDF screening	Separate PDF_SYSTEM_PROMPT with PDF files attached as file inputs
TF vs nTF	Same prompts/settings; different papers included at each stage
Models observed in code	gpt-4o; gpt-4.1-mini; google/gemini-3-flash-preview

1. Scope and source of appendix

This appendix documents the prompts, model identifiers, API interface, output schemas, and screening-batch rules implemented in the uploaded codebase. It is designed for direct inclusion as a manuscript appendix or supplementary file.

Code component	Role in appendix
prompts.py	Defines SYSTEM_PROMPT, PDF_SYSTEM_PROMPT, and review-specific criteria blocks
llm.py	Active model/API wrapper using OpenRouter chat completions; supports file inputs
gpt.py	Deprecated wrapper using OpenAI Responses API; retained as historical implementation
pdfs.py	Runs and parses full-text PDF screening, including retry batch sizes
util.py	Defines batching and parallel execution helpers

2. General system prompt

The general eligibility screening prompt was used for article-level screening based on title-only metadata or title-plus-abstract metadata. The same prompt text was used regardless of TF/nTF workflow.

SYSTEM_PROMPT

````
You are a scientist. You are given a set of inclusion and exclusion criteria and a list of articles. You
must decide
whether to include each article and explain why.
Try your best even if the abstract is missing.

The format should be as follows:

```
## ARTICLE 1

<reasoning>

INCLUDE or EXCLUDE
```

You must make a decision for **each** input and write it only once.
````

2.1 Article metadata template

For each article, the pipeline created a metadata block with title, authors, publication year, and abstract. Missing values were rendered as <Missing>.

Article block generated by _format_article()

```
## ARTICLE {index}

TITLE: <title or <Missing>>
AUTHORS: <authors or <Missing>>
PUBLISHED ON: <year or <Missing>>
ABSTRACT:
<abstract or <Missing>>
```

The input dataframe was expected to contain title, authors, year, and abstract. Capitalized variants of these column names were accepted and renamed internally.

3. Review-specific inclusion/exclusion criteria

The review-specific criteria were appended after the batch of article metadata. The wording below reproduces the criteria blocks implemented in the code. TF and nTF used the same criteria text for the same review.

3.1 Kuodi / post-COVID symptoms review

KUODI_CRITERIA

```
## INCLUSION CRITERIA

- Study type: Observational studies published in peer-reviewed literature.
- Participants: Individuals tested for SARS-CoV-2 by serological or molecular tests, with or without
control groups.
- Outcomes: Studies reporting physical and/or mental health symptoms, symptom clusters, new onset
symptoms, or ongoing symptoms following initial SARS-CoV-2 infection.
- Timeframe: No restriction on the time elapsed between infection and symptom reporting, as long as
participants had a prior SARS-CoV-2 infection.
- Language: Only studies published in English.
- Publication date: Studies published between January 2020 and August 2022.
- Geographic location: No restrictions applied.
```

3.2 Jumanah / MMR or HPV vaccination barriers and enablers review

ESSA-HADAD_CRITERIA

```
## INCLUSION CRITERIA
Studies that focus on barriers, enablers, or facilitators of MMR or HPV vaccination
Geography- global, Conducted in developed countries (mid-to-high income countries)
Study Design: Randomized controlled trials (RCTs), quasi-experimental, ecological, or observational cohort
studies•Study Design: Randomized controlled trials (RCTs), quasi-experimental, ecological, or
observational cohort studies
Peer reviewed articles
Articles published in English
Articles published since 2010
Population: children, adolescents, parents, health care workers in mid-to-high income countries
Excluding: interventions, studies from U.S. (case by case- depending on population looked at), studies
with health systems very different
```

3.3 Rhone / epidemiology of atopy in Israel review

RHONE_CRITERIA

```
## INCLUSION CRITERIA (Rhone):
Studies published in peer-reviewed journals
Studies conducted on Israeli population
Report on prevalence of:
Asthma
Atopic dermatitis
Food allergy
Allergic rhinitis
Provide data by age group, population subgroup, geographic region, or time period
Study types: Observational studies (cross-sectional, cohort, registry-based)
Published in English or Hebrew
Indexed in PubMed, Scopus or Web of Science
Excluding: Case reports, editorials, or reviews (unless they cite relevant primary data)
```

4. Stage-specific prompts

Screening stage	Prompt used	Input difference
Title screening	SYSTEM_PROMPT	Abstract field set to empty and displayed as <Missing>
Abstract screening	SYSTEM_PROMPT	Title, authors, year, and abstract supplied
Combined title-abstract screening	SYSTEM_PROMPT	Same effective implementation as abstract/title-abstract screening
Full-text PDF screening	PDF_SYSTEM_PROMPT	Review criteria supplied as text; PDF files attached separately

4.1 Title-screening prompt

Title screening used the general prompt. In the user message, the abstract field was masked as <Missing>.

Effective title-screening prompt

```
System message:
<SYSTEM_PROMPT>

User message:
## ARTICLE 1

TITLE: <article title>
AUTHORS: <article authors>
PUBLISHED ON: <publication year>
ABSTRACT:
<Missing>

## ARTICLE 2

TITLE: <article title>
AUTHORS: <article authors>
PUBLISHED ON: <publication year>
ABSTRACT:
```

(continued)

```
<Missing>

...

<review-specific inclusion/exclusion criteria>
```

4.2 Abstract-screening prompt

The implementation was not abstract-only in the strict sense. It supplied title, authors, year, and abstract, followed by the review-specific criteria.

Effective abstract/title-abstract prompt

```
System message:
<SYSTEM_PROMPT>

User message:
## ARTICLE 1

TITLE: <article title>
AUTHORS: <article authors>
PUBLISHED ON: <publication year>
ABSTRACT:
<article abstract>

## ARTICLE 2

TITLE: <article title>
AUTHORS: <article authors>
PUBLISHED ON: <publication year>
ABSTRACT:
```

(continued)

```
<article abstract>

...

<review-specific inclusion/exclusion criteria>
```

4.3 Combined title-abstract prompt

The combined title-abstract prompt was the default implementation when use_abstract=True. It was the same as the effective abstract-screening prompt above.

4.4 Full-text PDF prompt

Full-text screening used a separate system prompt. The review criteria were supplied as the text input, while PDF files were attached separately as file objects encoded as base64 data URLs.

PDF_SYSTEM_PROMPT

```
You are a scientist. You will be given several PDF files of scientific articles. You need to decide whether to
include each one of them in a study.

You will be given criteria to decide whether to include or to exclude.

For each article, please print the article's title, then your reasoning, then the decision whether to include or
exclude, in the following format:

```
TITLE
<article's title>

AUTHORS
<authors>

DOI
<doi ID>

PMID
<pubmed_id>
```
```

(continued)

```
REASONING
<reasoning why to include or to exclude>

DECISION
<one of INCLUDE or EXCLUDE>

TITLE
<next article's title>
```

To get the DOI ID, look for e.g., `doi:10.1016/j.vaccine.2014.05.058`, then print `10.1016/j.vaccine.2014.05.058` as the DOI ID. If there is no DOI ID, print `<missing>` as the ID.

If you find a PUBMED ID (aka PMID), include it under PMID. Otherwise, print `<missing>`.

etc.
You must do this for **each** input.

(continued)

```
Write out the decisions for all inputs **in the same order as they are given**.
```

5. Output schema and parsing rules

The implementation used free-text model outputs with strict parser expectations. Outputs that failed parsing or produced the wrong number of decisions were treated as missing for that batch.

5.1 Title / abstract / combined title-abstract output

Expected article-level output format

```
## ARTICLE 1

<reasoning>

INCLUDE

or

## ARTICLE 1

<reasoning>

EXCLUDE
```

Parsed field	Parsed type	Parser rule
reasoning	string	All lines under the article heading except the final decision line
decision	boolean or missing	INCLUDE maps to True; EXCLUDE maps to False; failures become missing values

- Article headings were expected to appear in sequential order.
- The parser accepted headings such as ARTICLE 1 and grouped headings such as ARTICLES 1-3, although the prompt requested one decision per article.

- If parsing failed, or if the number of parsed results did not equal the number of input articles, the batch was replaced with missing decision and reasoning values.

5.2 Full-text PDF output

Expected PDF-screening output format

```
TITLE
<article title>

AUTHORS
<authors>

DOI
<doi ID or <missing>>

PMID
<PMID or <missing>>

REASONING
<reasoning why to include or exclude>

DECISION
<INCLUDE or EXCLUDE>
```

Parsed field	Parsed type	Rule
title	string	Required; titles in a batch must be unique
authors	string	Required
doi	string	Required; <missing> is accepted as a non-empty value
pmid	string	Required; <missing> is accepted as a non-empty value
reasoning	string	Required
decision	boolean	Must be exactly INCLUDE or EXCLUDE; mapped to True or False

A PDF batch was treated as failed if any required field was missing, a decision was not exactly INCLUDE or EXCLUDE, titles were duplicated, or the number of parsed outputs did not match the number of PDFs in the batch.

6. Prompt refinement and prompt versioning

The code does not contain a formal prompt-version log, timestamped prompt-change history, or separate TF/nTF prompt variables. For reporting, the implemented prompts can be labelled as v1.

Observed change	Reproducibility interpretation
Deprecated wrapper	gpt.py is marked as the old implementation; llm.py is the active implementation
Provider/interface change	llm.py uses OpenRouter chat completions and supports optional PDF file inputs
Prompt centralization	prompts.py centralizes the general article prompt, PDF prompt, and review criteria
Title-only screening	Implemented by masking the abstract, not by changing prompt wording
PDF output expansion	PDF prompt requires title, authors, DOI, PMID, reasoning, and decision
TF/nTF prompt status	No separate TF or nTF prompt version exists in the code

7. Prompt version used for TF and nTF

Component	TF	nTF	Same or different?
Title screening prompt	SYSTEM_PROMPT v1; abstract masked if title-only	SYSTEM_PROMPT v1; abstract masked if title-only	Same
Abstract/title-abstract prompt	SYSTEM_PROMPT v1; title, authors, year, abstract supplied	SYSTEM_PROMPT v1; title, authors, year, abstract supplied	Same
Full-text PDF prompt	PDF_SYSTEM_PROMPT v1	PDF_SYSTEM_PROMPT v1	Same
Review-specific criteria	Same criteria block for the target review	Same criteria block for the target review	Same
Output schema and parser	Same parser and validation rules	Same parser and validation rules	Same
Model/API settings	Same model interface and settings within the comparison	Same model interface and settings within the comparison	Same
Papers entering each step	Records/papers included under the TF step workflow	Records/papers included under the nTF step workflow	Different

Operational definition

At each downstream stage, the workflow screened only the papers that had been included and carried forward from the preceding stage of that same workflow. This propagation rule explains the TF/nTF difference without requiring any prompt change.

8. Model and API settings

The active implementation defines model identifiers as constants and passes the chosen model dynamically at runtime. The code alone does not provide a complete run log mapping every experiment to a model argument, but it does identify the model versions available/used in the implementation.

Model constants in llm.py

```
GPT_FULL = 'gpt-4o'
GPT_MINI = 'gpt-4.1-mini'
GEMINI_3_FLASH = 'google/gemini-3-flash-preview'
```

Model family	Model identifier in code	Reporting name
OpenAI / ChatGPT	gpt-4o	GPT-4o
OpenAI / ChatGPT	gpt-4.1-mini	GPT-4.1-mini
Google Gemini	google/gemini-3-flash-preview	Gemini 3 Flash preview

The old deprecated wrapper also defined Models.FULL as gpt-4o and Models.MINI as gpt-4.1-mini, with the old check_eligibility default set to Models.MINI.

8.1 API interface

Active API call structure

```
POST https://openrouter.ai/api/v1/chat/completions

messages = [
    {'role': 'system', 'content': instructions},
    {'role': 'user', 'content': user_content},
]
```

- For non-PDF screening, user_content contained one text item with the article batch and criteria.

- For PDF screening, user_content contained the review criteria text plus one file object per PDF.
- Each PDF was encoded as a base64 data URL before being sent as a file input.

8.2 Generation parameters not explicitly set

The uploaded code did not explicitly set the following generation parameters. These should therefore be reported as provider defaults unless external run logs specify otherwise.

Parameter	Status in uploaded code
temperature	Not explicitly set
top_p	Not explicitly set
max_tokens	Not explicitly set
seed	Not explicitly set
presence_penalty	Not explicitly set
frequency_penalty	Not explicitly set
stop sequences	Not explicitly set
response_format / JSON mode	Not explicitly set

9. Batch size rules for PDF screening

Full-text PDF screening was implemented as a retry cascade. The pipeline first attempted larger PDF batches, then retried unresolved articles with smaller batch sizes.

PDF retry cascade in EvalProcess.evaluate()

```
for batch_size in [5, 3, 1]:
    print(f'Running batches of size {batch_size}')
    self._run_on_missing_items(batch_size)
```

Attempt	Batch size	Applied to
1	5 PDFs	All PDFs not yet parsed
2	3 PDFs	Only PDFs still missing parsed title/results
3	1 PDF	Any remaining unresolved PDFs

Only items without a parsed title were retried at each smaller batch size. If a batch produced an API error, parser error, or a number of parsed outputs different from the number of input PDFs, that batch returned no results and the affected PDFs remained eligible for the next retry pass.